

IA embarquée pour la perception des véhicules autonomes en logistique hospitalière

État des lieux, état de l'art et choix technologiques

Encadrement	Moïse DJOKO-KOUAM
Équipe projet	<i>à compléter</i>
Version	1.0 — document de travail
Date	Septembre 2026
Mots clés	Intelligence artificielle · Perception · Fusion · Suivi · Embarqué

Livrables couverts : **analyse des besoins et des contraintes · état de l'art et comparaison des modèles · choix argumenté du modèle** (proposition, à confirmer par les mesures du protocole expérimental).

Sommaire

Partie I — Cadrage du projet

1. Ce que dit le cahier des charges
2. Contrainte matérielle : ce que le Raspberry Pi 5 autorise
3. Décomposition fonctionnelle proposée
4. Budget temps réel — d'où vient la cible de latence
5. Contraintes spécifiques au milieu hospitalier
6. Critères de réussite proposés (mesurables)
7. Risques identifiés (extrait — à reprendre dans le plan de gestion des risques)
8. Ce qui reste à arbitrer avec l'encadrant

Partie II — État des lieux de l'existant

- Préambule — démarche et objet du document
1. État des lieux — les systèmes en production
 2. État des lieux — la recherche académique
 3. État des lieux — les projets embarqués bas coût
 4. Les briques logicielles standard du domaine
 5. Synthèse technologique de l'existant
 6. Les six enseignements transversaux
 7. Positionnement de notre projet dans ce paysage
 8. L'entonnoir de décision
 9. Traçabilité des choix — de l'observation à la décision
 10. Écarts assumés avec l'existant et risques associés
 11. Sources

Partie III — État de l'art et comparaison des modèles

1. Méthodologie de la comparaison
2. F1 — Détection d'objets 2D
3. F2 — Segmentation de l'espace libre
4. F3 — Estimation de la distance
5. F4 — Suivi multi-objets
6. F5 — Fusion, état sémantique et niveau de confiance
7. Chaîne d'exécution : formats, runtimes et optimisation
8. Option matérielle : accélérateurs pour Raspberry Pi 5
9. Grille de décision multicritère
10. Recommandation argumentée
11. Protocole de validation expérimentale
12. Sources

Cadrage du projet

PRI 2026-2027 — Projet 2 : IA embarquée pour la perception des véhicules autonomes en logistique hospitalière

1. Ce que dit le cahier des charges

1.1 Objectif et problématique

Élément	Énoncé
Objectif	Transformer des informations de perception hétérogènes de l'environnement en une représentation structurée et exploitable de l'environnement du VA
Problématique	Comment concevoir une IA de perception performante, frugale et apte au déploiement embarqué ?
Objectif final	Démontrer la faisabilité d'une IA de perception locale, frugale et embarquable , produisant des données structurées, fiables et exploitables par une IA de décision

Le point structurant, souvent sous-estimé : **le module de perception n'est pas le produit final**. Son client est une *IA de décision* (hors périmètre de ce projet). Le livrable réel est donc un **contrat d'interface** — un flux de données structurées, horodatées et accompagnées d'un niveau de confiance — et non « un détecteur qui marche ».

1.2 Position dans l'architecture du VA

Le module de perception assure l'*acquisition, la représentation, la diffusion et la synchronisation* des informations contextuelles provenant de trois sources :

- **Environnement proche** : caméra, LiDAR, sonar, IMU, odométrie
- **Environnement lointain** : informations d'infrastructure
- **Autres VA**

Parc de capteurs annoncé : **caméra, LiDAR, IMU, moteurs, sonar, BIR, RFID, GPS**.

⚠ Constat à arbitrer dès le lancement. Le GPS est inopérant en intérieur hospitalier (couloirs, sous-sols, blocs). Le BIR et le RFID relèvent de la **localisation symbolique** (identification de zone, de porte, de chariot) et non de la perception géométrique : ils alimentent la représentation de l'environnement mais ne passent pas par le réseau de neurones. La proposition de périmètre défendue en §3 est donc : **caméra + LiDAR + sonar + IMU/odométrie** pour l'IA de perception, RFID/BIR en entrées symboliques fusionnées en aval.

1.3 Sorties attendues (« données de suivi »)

Le cahier des charges énumère les sorties suivantes — c'est la spécification de l'interface :

#	Sortie	Nature	Brique fonctionnelle
a	Obstacle détecté / position / distance	Géométrique + classe	Détection + distance
b	Personne présente	Classe + comptage	Détection
c	Couloir dégagé ou obstrué	État booléen/ordinal	Espace libre
d	Ascenseur disponible	État sémantique	Détection + état
e	Trajectoire libre ou bloquée	État booléen	Espace libre + suivi
f	Niveau de confiance / incertitude	Scalaire [0,1]	Transverse

La sortie **(f)** est la plus exigeante et la plus discriminante pour le choix du modèle : elle impose une **calibration des scores** (un score de 0,8 doit signifier 80 % de justesse empirique), ce qui n'est vrai pour aucun détecteur sorti de l'entraînement standard. Elle est traitée en détail dans l'état de l'art (§6 du document de comparaison).

1.4 Travail demandé

1. Analyser les besoins et les contraintes
2. **Étudier plusieurs modèles d'IA**
3. **Choisir et justifier le modèle retenu**
4. Constituer / préparer les données
5. Entraîner et optimiser le modèle
6. Implémenter et tester sur **Raspberry Pi 5**

1.5 Livrables techniques

- État de l'art et comparaison des modèles ← *couvert par le présent lot*
- Choix argumenté du modèle
- Jeu de données et protocole de préparation
- Modèle entraîné et optimisé
- Implémentation embarquée
- Protocole et résultats expérimentaux

1.6 Livrables de gestion de projet (communs à tous les PRI)

Cahier des charges / expression des besoins · objectifs, périmètre et critères de réussite · planning et jalons · répartition des rôles · analyse des risques et plan d'actions · suivi d'avancement (réunions, comptes rendus, indicateurs) · suivi des ressources (matériel, logiciels, budget, temps) · gestion des évolutions · rapport final · présentation & démonstration.

1.7 Critères d'évaluation du système

Trois familles, qui forment directement l'ossature de la grille de décision :

Famille	Critères
Performance	Précision · Robustesse · Complexité
Frugalité	Taille du modèle · RAM / CPU · Énergie
Embarqué	Temps d'inférence · Utilisation CPU · Temps réel

2. Contrainte matérielle : ce que le Raspberry Pi 5 autorise

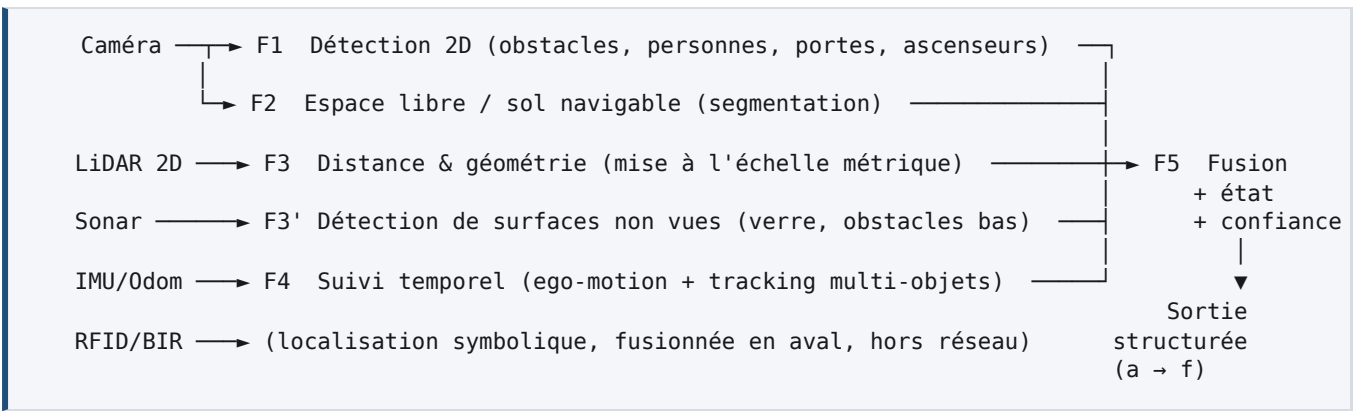
Caractéristique	Valeur	Conséquence pour le choix du modèle
SoC	Broadcom BCM2712	—
CPU	4 × Arm Cortex-A76 @ 2,4 GHz	Pas de GPU exploitable pour l'inférence, pas de NPU intégré
SIMD	NEON, avec instructions dot-product (SDOT/UDOT)	L'INT8 est réellement accéléré ($\approx \times 2$ mesuré sur A76), contrairement au Cortex-A53 du Pi 3 où l'INT8 <i>ralentit</i>
RAM	4 / 8 / 16 Go LPDDR4X	Budget mémoire confortable, ce n'est pas le facteur limitant
Extension	PCIe (AI Kit / AI HAT+ Hailo-8L, 13 TOPS)	Voie de secours si le CPU seul ne tient pas le budget
Puissance	~3-4 W au repos, ~7-12 W en charge	Contrainte batterie du VA ; le SoC <i>throttle</i> sans dissipation active

Deux conséquences directes :

1. **Toute l'inférence se fait sur 4 cœurs CPU ARM.** Les chiffres de latence publiés sur GPU (T4, A100) ou sur Jetson sont **non transposables** — ils servent uniquement à classer les modèles entre eux, jamais à dimensionner.
2. **Le refroidissement fait partie du protocole expérimental.** Un Pi 5 sans ventilateur perd 20 à 40 % de ses performances après quelques minutes de charge continue. Toute mesure de latence non accompagnée d'une mesure de température et d'une durée de chauffe est invalide.

3. Décomposition fonctionnelle proposée

Le sujet « IA de perception » n'est pas monolithique. Pour rendre le choix des modèles traçable, on le décompose en cinq briques, chacune avec ses propres candidats :



Brique	Rôle	Sorties couvertes
F1 Détection 2D	Localiser et classer obstacles, personnes, portes, ascenseurs, chariots	a, b, d
F2 Espace libre	Segmenter le sol navigable devant le VA	c, e
F3 Distance	Passer du pixel au mètre (LiDAR 2D, sonar, ou profondeur monoculaire)	a, e
F4 Suivi	Stabiliser les détections dans le temps, estimer les vitesses	b, e, f
F5 Fusion & décision d'état	Produire la sortie structurée + le niveau de confiance	c, d, e, f

C'est cette décomposition qui structure l'état de l'art : **un modèle est comparé aux autres candidats de sa brique**, pas à l'ensemble du champ.

4. Budget temps réel — d'où vient la cible de latence

C'est le calcul qui transforme « temps réel » (exigence qualitative du cahier des charges) en un seuil chiffré, opposable, utilisable comme critère d'élimination.

Hypothèses de service (à valider avec l'encadrant) :

Paramètre	Valeur retenue	Justification
Vitesse de croisière du VA en couloir	1,0 m/s	Vitesse usuelle d'un AMR hospitalier en zone partagée avec des piétons
Vitesse d'un piéton croisé de face	1,4 m/s	Vitesse de marche moyenne
Vitesse de rapprochement pire cas	2,4 m/s	Somme des deux
Décélération confortable (charge fragile)	1,0 m/s ²	Un chariot hospitalier peut transporter des prélèvements, des repas, des médicaments
Marge de sécurité	0,5 m	Distance d'arrêt résiduelle

Chaîne de latence à budgéter :

acquisition → pré-traitement → inférence → post-traitement → fusion → publication
 ~15 ms ~10 ms ??? ~10 ms ~5 ms ~5 ms

Le budget hors inférence est d'environ **45 ms**. En visant une cadence de **10 Hz** (100 ms par cycle, cadence standard d'une boucle de perception d'AMR), il reste :

Budget d'inférence cible : ≤ 50 ms par image, soutenu, à température stabilisée

soit une cadence de perception de 10 Hz et un déplacement de **10 cm par cycle** à 1 m/s.

Vérification de cohérence — distance de réaction totale à 2,4 m/s de rapprochement : latence perception (0,10 s) + latence décision/commande (0,15 s estimé) = 0,25 s → 0,60 m parcourus, plus la distance d'arrêt ($2,4^2/(2 \times 1,0) = 2,88$ m) et la marge (0,5 m) ≈ **4,0 m de portée de détection utile**. C'est compatible avec la portée d'une caméra grand angle en couloir éclairé et avec un LiDAR 2D d'intérieur (8-12 m).

Seuils de décision retenus pour la comparaison :

Niveau	Latence d'inférence (RPI 5, 4 threads, à chaud)	Verdict
● Confortable	≤ 50 ms (≥ 20 Hz)	Marge pour la fusion et le suivi
● Acceptable	50 – 100 ms (10-20 Hz)	Tenable, sans brique supplémentaire coûteuse
● Limite	100 – 200 ms (5-10 Hz)	Impose de réduire la vitesse du VA ou d'ajouter un accélérateur
● Éliminatoire	> 200 ms (< 5 Hz)	Incompatible avec un environnement partagé avec des piétons

5. Contraintes spécifiques au milieu hospitalier

Ces contraintes sont **discriminantes** : elles écartent des modèles qui seraient excellents sur COCO.

Contrainte	Impact technique	Conséquence sur le choix
Sols réfléchissants (linoléum ciré, carrelage)	Reflets interprétés comme des obstacles ; faux positifs de segmentation	Favorise une segmentation de l'espace libre <i>validée</i> par le LiDAR plutôt que purement visuelle
Surfaces vitrées (portes de service, sas)	Invisibles au LiDAR et à la caméra	Rend le sonar non optionnel — c'est sa justification principale
Éclairage hétérogène (couloirs, néons, veilleuses de nuit)	Chute de performance en basse lumière	Impose une augmentation de données photométrique agressive et un jeu de test nocturne dédié
Encombrement mobile (brancards, chariots, perfusions, déambulateurs)	Classes absentes de COCO	Impose un jeu de données propre ; écarte tout modèle non ré-entraînable facilement
Personnes en position non debout (patient au sol, brancard)	La classe « person » COCO est biaisée vers le piéton debout	Impose une collecte incluant ces cas ; critère de robustesse
Sécurité des personnes (ISO 3691-4 / ISO 13482)	Un faux négatif sur « personne » dégrade fortement le service ; un faux positif est seulement une gêne	Le rappel (recall) sur la classe « personne » prime sur la précision — asymétrie à inscrire dans la fonction de coût et dans le protocole de test. ⚠ Voir §4.2 de l'état des lieux (partie II) : ISO 3691-4 exige un niveau PLd pour la détection de personnes, qu'aucun réseau de neurones ne peut atteindre. Dans tous les systèmes en production, la sécurité repose sur un scanner laser certifié, et l'IA sur la compréhension de la scène. Notre module est un démonstrateur de perception, pas un organe de sécurité
Confidentialité (RGPD, secret médical)	Interdiction de faire transiter des images de patients	Confirme et <i>justifie</i> l'exigence de traitement 100 % local du cahier des charges ; impose une politique de stockage du jeu de données

6. Critères de réussite proposés (mesurables)

Proposition à faire valider en réunion de lancement — elle transforme les critères qualitatifs du cahier des charges en seuils vérifiables.

#	Critère	Seuil	Méthode de vérification
C1	Rappel sur la classe « personne »	≥ 95 % @ IoU 0,5, sur le jeu de test hospitalier	Évaluation hors ligne
C2	mAP@0,5 toutes classes	≥ 60 % sur le jeu de test propre	Évaluation hors ligne
C3	Latence d'inférence médiane	≤ 50 ms sur RPi 5, 4 threads	Mesure sur 1 000 images, après 10 min de chauffe
C4	Latence au 95 ^e centile	≤ 100 ms	Même mesure (garantit le déterminisme)
C5	Empreinte mémoire du processus	≤ 1 Go RSS	psutil / /proc pendant la boucle
C6	Taille du modèle déployé	≤ 15 Mo	Fichier sur disque
C7	Puissance système en fonctionnement	≤ 8 W moyens	Wattmètre USB-C en ligne
C8	Stabilité thermique	Pas de throttling sur 30 min	vcgencmd get_throttled
C9	Calibration de la confiance	ECE ≤ 0,05	Diagramme de fiabilité
C10	Disponibilité de la sortie structurée	10 Hz soutenus, sans trou > 300 ms	Horodatage du flux de sortie

7. Risques identifiés (extrait — à reprendre dans le plan de gestion des risques)

Risque	Gravité	Probabilité	Parade
Le jeu de données hospitalier ne peut pas être collecté (accès, RGPD)	Élevée	Moyenne	Démarrer sur données publiques d'intérieur + collecte dans les couloirs de l'école comme substitut ; anonymisation par floutage à la source
Le CPU seul ne tient pas le budget de 50 ms	Élevée	Moyenne	Réduction de résolution (640 → 416 → 320), INT8, puis repli sur AI Kit Hailo-8L
Throttling thermique en démonstration	Moyenne	Élevée	Dissipateur actif obligatoire dès la phase de mesure
Modèle excellent sur COCO mais médiocre sur les classes hospitalières	Élevée	Élevée	Critère de transférabilité intégré à la grille de comparaison (§7 du document d'état de l'art)
Dérive de périmètre vers l'IA de décision	Moyenne	Moyenne	Contrat d'interface figé tôt ; la décision est hors périmètre
Capteurs (LiDAR, sonar) indisponibles ou livrés tard	Moyenne	Moyenne	Architecture en briques : F1 seule est démontrable sans LiDAR

8. Ce qui reste à arbitrer avec l'encadrant

1. **Périmètre capteurs** : GPS et RFID/BIR sont-ils réellement à traiter, ou hors périmètre en intérieur ? (§1.2)
 2. **Vitesse nominale du VA** — elle conditionne tout le budget de latence. (§4)
 3. **Modèle de LiDAR disponible** : 2D (type RPLIDAR) ou 3D ? L'écart de complexité entre les deux est d'un ordre de grandeur.
 4. **Accès terrain** pour la collecte du jeu de données, et cadre RGPD associé.
 5. **L'AI Kit (Hailo-8L) est-il autorisé** au budget, ou la démonstration doit-elle rester sur CPU nu ? (Cela change la réponse de l'état de l'art.)
 6. **Format du contrat d'interface** avec l'IA de décision : ROS 2 ? JSON sur MQTT ? Cadence fixe ou événementielle ?
-

État des lieux de l'existant

PRI 2026-2027 — Projet 2 : IA embarquée pour la perception des véhicules autonomes en logistique hospitalière
Version 1 — septembre 2026

Préambule — démarche et objet du document

Avant de choisir une technologie, il faut savoir ce que d'autres ont déjà fait du même problème. Ce document répond à trois questions, dans cet ordre :

1. **Que fait-on aujourd'hui ?** Quels systèmes de logistique hospitalière autonome existent réellement, en production comme en recherche, et jusqu'où vont-ils ?
2. **Avec quelles technologies ?** Quels capteurs, quels modèles, quelles architectures logicielles, quel matériel de calcul ces systèmes emploient-ils concrètement ?
3. **Qu'est-ce que cela implique pour nous ?** Comment ces observations, croisées avec nos contraintes propres, conduisent-elles — de façon traçable — aux technologies retenues ?

La démarche suivie correspond à la méthodologie en cinq étapes identifiée dans la littérature récente sur le déploiement de réseaux de neurones embarqués : *définition du besoin* → *sélection du modèle* → *optimisation* → *alignement matériel* → *déploiement*. Le présent document couvre la première étape et amorce la deuxième ; l'état de l'art des modèles (partie III) traite la deuxième et la troisième.

1. État des lieux — les systèmes en production

La logistique hospitalière autonome n'est pas un domaine émergent : c'est un marché mature, avec des flottes déployées depuis plus de quinze ans et des centaines de milliers de livraisons effectuées. Quatre systèmes structurent le paysage.

1.1 Aethon TUG (États-Unis, déployé depuis ~2004)

Le vétéran du domaine, présent dans plusieurs centaines d'hôpitaux nord-américains. Transport de médicaments, de repas, de linge et de prélèvements.

Aspect	Ce qui est mis en œuvre
Capteurs	Suite multi-capteurs : LiDAR, ultrasons, infrarouge , et depuis les générations récentes une caméra de profondeur Intel RealSense D435
Cœur de la navigation	SLAM — construction et mise à jour continue d'une carte de l'environnement
Apport de la caméra	Explicitement introduite pour « voir des obstacles que les ultrasons et l'infrarouge ne peuvent pas détecter », par calcul de profondeur
Intégration bâtiment	Communication avec ascenseurs et portes automatiques

Ce qu'il faut en retenir : la caméra est arrivée **en dernier**, en complément d'une base télémétrique déjà fonctionnelle. Elle résout un problème résiduel de couverture, elle ne constitue pas le socle de la perception.

1.2 Panasonic HOSPI (Japon, déployé depuis ~2013)

Le système le plus intéressant pour nous, parce que c'est le seul du panel dont la **sécurité est certifiée** et documentée publiquement.

Aspect	Ce qui est mis en œuvre
Capteurs	Multiples capteurs conçus pour détecter des obstacles « de formes et même de hauteurs variées » en intérieur
Navigation	Carte pré-programmée du bâtiment + reconnaissance de l'environnement par capteurs
Ascenseurs et portes	Coordination par le réseau — le robot appelle l'ascenseur et commande les portes automatiques via une liaison réseau
Infrastructure	Aucune modification du bâtiment : ni fils, ni capteurs encastrés, ni bandes électromagnétiques au sol
Sécurité	Certifié ISO 13482 , et première certification sous les nouvelles normes JIS
Supervision	Remontée de position en continu vers un centre de contrôle

Ce qu'il faut en retenir — deux points majeurs :

1. **La disponibilité de l'ascenseur n'est pas un problème de vision.** Le système industriel de référence la résout par une **requête réseau**, qui donne une information exacte, instantanée et fiable à 100 %. Aucun modèle de perception ne peut égaler cela.
2. Le refus de modifier l'infrastructure est un choix commercial assumé (coût d'installation), pas une contrainte technique. Il valide en revanche l'approche « perception embarquée autonome ».

1.3 Relay Robotics (ex-Savioke, États-Unis)

Robots de livraison pour hôtels, bureaux et hôpitaux ; gammes Relay+, Relay2. Positionnement explicite sur les environnements publics **encombrés**.

Aspect	Ce qui est mis en œuvre
Navigation	Autonomie complète en environnement public fréquenté
Ascenseurs	Le robot appelle l'ascenseur, sélectionne sa destination, entre et sort seul . La communication est configurée à l'installation — pas d'intégration lourde du bâtiment
Déploiement	Argument « Rapid Install » : opérationnel en jours, pas en mois

Ce qu'il faut en retenir : la rapidité de mise en service est un argument commercial de premier plan. Cela plaide, pour nous aussi, contre toute solution exigeant une longue campagne d'annotation propre à chaque site.

1.4 Diligent Robotics Moxi (États-Unis)

Le plus avancé en perception, parce que c'est le seul à **manipuler** des objets et à interagir socialement avec le personnel soignant.

Aspect	Ce qui est mis en œuvre
Capteurs	Capteurs multiples, caméras, algorithmes d'IA
Fonctions	Navigation autonome + manipulation dextre + interaction sociale avec le personnel

Ce qu'il faut en retenir : c'est le seul du panel pour lequel l'IA de perception visuelle est réellement centrale — et c'est précisément parce qu'il doit **saisir des objets** et **comprendre des personnes**, pas seulement les éviter. Le niveau de perception requis est fonction de la tâche, pas du prestige technologique.

2. État des lieux — la recherche académique

2.1 ORB — Operating Room Bot (Carnegie Mellon, IEEE CASE 2025)

Le travail académique récent le plus proche de notre domaine : automatisation de la logistique **au sein même du bloc opératoire**.

Aspect	Ce qui est mis en œuvre
Plateforme	Fetch (base mobile + bras 7 axes + préhenseur à ventouse), capteurs multimodaux
Logiciel	ROS 2 , architecture modulaire pilotée par arbres de comportement (<i>behavior trees</i>)
Perception	Chaîne combinant YOLOv7 + SAM 2 (Segment Anything) + Grounding DINO
Planification	cuRobo , optimisation de trajectoire parallélisée accélérée GPU
Résultats	80 % de succès en récupération de fournitures, 96 % en réapprovisionnement

Ce qu'il faut en retenir : l'état de l'art académique en perception hospitalière empile **trois grands modèles de vision** et exige un **GPU**. C'est la démonstration, par l'exemple, de ce que notre projet ne peut pas faire — et donc la mesure exacte de l'écart à franchir. La comparaison est éclairante : la chaîne d'ORB représente plusieurs centaines de millions de paramètres, là où notre budget se compte en **quelques millions**.

2.2 Ce que disent les revues de synthèse récentes

Trois enseignements convergents ressortent des revues 2025-2026 sur la perception embarquée en robotique mobile :

- **Le pipeline est unifié et standardisé** : acquisition → perception → localisation et cartographie → prédiction → planification → contrôle. Notre projet occupe la case « perception », ce qui confirme la nécessité d'un **contrat d'interface** propre avec l'aval.
- **La méthodologie de déploiement embarqué est stabilisée** en cinq étapes (besoin → sélection → optimisation → alignement matériel → déploiement), avec un triptyque d'optimisation récurrent : **élagage, quantification, architectures légères**. C'est exactement la démarche du §7 de notre état de l'art des modèles — notre approche est donc alignée sur la pratique du domaine.
- **Le multi-tâche à dorsale partagée s'impose sur l'embarqué**. Des travaux récents montrent qu'une dorsale unique de type ResNet-18 servant plusieurs têtes offre le meilleur compromis charge/représentation pour un robot autonome déployé sur cible contrainte. **C'est la validation académique directe de notre principe « un seul réseau dans la boucle ».**

3. État des lieux — les projets embarqués bas coût

C'est la catégorie la plus comparable à la nôtre : prototypes académiques sur Raspberry Pi. Elle donne les ordres de grandeur réellement atteints, loin des annonces commerciales.

Projet	Matériel	Modèle	Performance rapportée	Lecture critique
Robot de surveillance autonome	Raspberry Pi 5	YOLOv8	6-8 FPS	Cohérent avec nos tableaux (§2.3 de l'état de l'art). Hors de notre budget de 10 Hz
Navigation en environnement dynamique	Raspberry Pi	YOLOv8	90 % de succès de navigation ; précision 87 % (objets mobiles) / 81 % (fixes)	Montre qu'une navigation utile est atteignable avec une perception imparfaite
Prototype ADAS embarqué	Raspberry Pi 3	YOLOv12n personnalisé	Temps réel revendiqué après optimisation	Confirme que l'optimisation, pas l'architecture, fait la faisabilité
Robot guide	Raspberry Pi 4B	YOLOv10	85 % de précision, « 57 FPS », erreur de distance 0,6 m (ultrasons stéréo)	⚠ Chiffre à considérer avec prudence : 57 FPS sur un Pi 4B est incompatible avec toutes les autres mesures publiées. Résolution très réduite, ou FPS de la boucle et non de l'inférence

Ce qu'il faut en retenir — trois points :

1. **Le régime réellement observé sur Raspberry Pi est de 6 à 8 FPS avec un YOLO non optimisé.** C'est notre point de départ honnête, et il est **sous** notre cible.
2. **Aucun de ces travaux ne publie de mesure rigoureuse** : ni p95, ni température, ni consommation, ni durée de chauffe. C'est une faiblesse méthodologique générale du domaine — et donc un espace où notre projet peut apporter une contribution réelle (§7.3).
3. **Tous s'arrêtent à la détection.** Aucun ne produit de représentation structurée avec niveau de confiance calibré — ce que notre cahier des charges demande explicitement.

4. Les briques logicielles standard du domaine

4.1 ROS 2 et Nav2 : la représentation de référence

Quasiment tous les systèmes cités s'appuient sur ROS / ROS 2. La représentation de l'environnement y est normalisée : la **costmap 2D en couches**.

Couche	Rôle	Source de données
<i>Static layer</i>	Obstacles connus et permanents (murs)	Carte SLAM pré-établie
<i>Obstacle layer</i>	Obstacles dynamiques, suivi en 2D	Scans laser, nuages de points
<i>Voxel layer</i>	Obstacles suivis en 3D , puis projetés en 2D pour l'inflation	Nuages de points
<i>Inflation layer</i>	Marge de sécurité autour des obstacles	Calculée

Ce qu'il faut en retenir : le format de sortie attendu par l'aval est **déjà normalisé**. Notre « représentation structurée » a tout intérêt à être compatible avec ce modèle en couches plutôt qu'à

inventer un format propriétaire. C'est un argument fort pour le contrat d'interface, et cela rend le travail réutilisable au-delà du PRI.

4.2 Les normes de sécurité — l'enseignement le plus important du document

Norme	Portée	Exigence clé
ISO 3691-4	Chariots sans conducteur / AMR industriels	La détection de personnes doit être assurée par un système de niveau de performance PLd (ISO 13849) : redondance, détection de défaut, surveillance indépendante. Un « test de détection des personnes » est explicitement normalisé
ISO 13482	Robots de service et d'assistance à la personne (norme du HOSPI)	Sécurité des robots serviteurs mobiles ; révision en cours

Conséquence architecturale majeure

Aucun réseau de neurones ne peut être certifié PLd. Un modèle appris est statistique et non déterministe : il est structurellement incapable de fournir la garantie exigée par ISO 3691-4.

C'est pourquoi, dans **tous** les systèmes commerciaux du §1, la sécurité des personnes repose sur un **scanner laser de sécurité certifié**, câblé à l'arrêt d'urgence — et **jamais sur l'IA de perception**. L'IA sert à *comprendre* la scène (anticiper, ralentir en douceur, choisir un itinéraire, décider qu'un couloir est praticable) ; le matériel certifié sert à *garantir l'arrêt*.

Ceci nuance une formulation de notre document de cadrage. Nous y écrivions qu'un faux négatif sur la classe « personne » est une faute grave, en traitant implicitement l'IA comme l'organe de sécurité. La réponse du domaine n'est pas d'exiger un modèle parfait — c'est de **placer l'IA derrière une couche de sécurité déterministe**. Le rappel sur la classe « personne » reste notre critère de performance prioritaire (qualité du service, fluidité, anticipation), mais il n'est plus le dernier rempart. C'est une correction importante : elle rend le projet **défendable**, là où prétendre garantir la sécurité des patients avec un YOLO quantifié sur Raspberry Pi serait indéfendable devant un jury.

5. Synthèse technologique de l'existant

Tableau de synthèse — qui utilise quoi, et sur quel matériel de calcul.

Système	Télémétrie	Vision	Ultrasons	Calcul	Modèles d'IA	Ascenseur
Aethon TUG	LiDAR + IR	RealSense D435 (profondeur)	✓	Embarqué x86 (non publié)	Non publié ; SLAM au cœur	Réseau
Panasonic HOSPI	Capteurs multiples multi-hauteurs	non détaillée	✓	Embarqué (non publié)	Évitement + carte pré-établie	Réseau
Relay Robotics	LiDAR + profondeur	✓	✓	Embarqué (non publié)	Navigation sociale	Réseau
Diligent Moxi	LiDAR	Caméras multiples	—	Embarqué haute performance	IA de perception + manipulation	Réseau
ORB (CMU)	Capteurs multimodaux	✓	—	GPU	YOLOv7 + SAM 2 + Grounding DINO	n/a
Projets RPi académiques	Souvent aucune	Caméra seule	Parfois	Raspberry Pi 3/4/5	YOLOv8 / v10 / v12n	n/a
→ Notre projet	LiDAR 2D	Caméra	Sonar	Raspberry Pi 5, CPU nu	YOLO26n + ByteTrack	Réseau recommandé

Observations issues du tableau

- **La télémétrie est universelle en production ; la vision est un complément.** Aucun système déployé ne fonde sa navigation sur la caméra seule.
- **Les ultrasons survivent** dans la quasi-totalité des systèmes commerciaux, malgré leur âge. Raison unique et suffisante : **le verre**. Cela confirme leur présence à notre cahier des charges.
- **Le fossé de calcul est béant** entre la recherche (GPU) et les prototypes bas coût (Pi). Notre projet se place délibérément du côté contraint — c'est là que réside sa contribution.
- **Personne ne détecte l'ascenseur par vision.** Tous passent par le réseau.

6. Les six enseignements transversaux

Ce que l'état des lieux nous apprend, et qui oriente directement les choix du §8.

#	Enseignement	Source de l'observation
E1	La géométrie d'abord, la sémantique ensuite. Tous les systèmes en production fondent leur navigation sur la télémétrie (LiDAR + ultrasons) et ajoutent la vision par-dessus, pour ce que la télémétrie ne voit pas	TUG, HOSPI, Relay
E2	La sécurité des personnes ne passe pas par l'IA , mais par du matériel certifié PLd. L'IA apporte le confort, l'anticipation et la sémantique	ISO 3691-4, ISO 13482 / HOSPI
E3	Ce qui est connu par le réseau ne doit pas être deviné par un capteur. L'état de l'ascenseur est obtenu par requête, pas par perception	HOSPI, Relay
E4	L'empilement de modèles est réservé au GPU. Sur cible contrainte, la pratique validée est la dorsale unique multi-tâches	ORB vs travaux ResNet-18 embarqués
E5	Sur Raspberry Pi, le régime naturel est 6-8 FPS. Atteindre 10 Hz+ exige une chaîne d'optimisation explicite (quantification, élagage, architectures légères)	Projets RPi académiques ; revues de déploiement embarqué
E6	Le format de sortie est déjà normalisé (costmap en couches ROS 2). Inventer un format propriétaire serait une régression	Nav2

7. Positionnement de notre projet dans ce paysage

7.1 Ce que notre projet n'est pas

Dire ce qu'on ne fait pas est aussi structurant que dire ce qu'on fait :

- **Ce n'est pas un concurrent du TUG ou du HOSPI.** Ceux-ci sont des produits complets (navigation, flotte, sécurité certifiée, intégration bâtiment). Nous traitons **une brique** : la perception.
- **Ce n'est pas un projet de navigation.** La décision et la planification sont explicitement hors périmètre (le cahier des charges parle d'« une IA de décision » comme consommateur aval).
- **Ce n'est pas une démonstration de performance maximale.** ORB fait mieux que ce que nous ferons, avec un GPU. Reproduire ORB en moins bien n'aurait aucune valeur.

7.2 Ce qui rend notre projet spécifique

Le sujet pose une question que **ni les produits ni les travaux académiques cités ne traitent frontalement** :

Jusqu'où peut-on descendre en coût et en consommation tout en produisant une représentation structurée, fiable et **assortie d'un niveau de confiance**, exploitable par une IA de décision ?

Les industriels ont résolu le problème **en y mettant le matériel nécessaire**. Les académiques l'ont résolu **en y mettant un GPU**. Les prototypes Raspberry Pi, eux, s'arrêtent à la détection brute, sans structuration ni quantification de l'incertitude. **La case est vide, et c'est la nôtre.**

7.3 Contributions revendiquées

#	Contribution	Vide qu'elle comble
C-1	Protocole de mesure rigoureux sur Raspberry Pi 5 : médiane <i>et</i> p95, température, throttling, consommation, durée de chauffe documentée	Aucun des travaux RPi recensés ne publie ces éléments (§3)
C-2	Sortie à confiance calibrée (ECE mesuré, temperature scaling)	Aucun prototype embarqué recensé ne quantifie son incertitude
C-3	Fusion caméra / LiDAR 2D à coût CPU nul pour la distance métrique	La littérature RPi utilise soit la vision seule, soit des ultrasons peu précis (0,6 m d'erreur)
C-4	Courbe de compromis résolution / rappel documentée en contexte couloir	Non documenté : les benchmarks publics sont établis sur COCO, dont la statistique d'échelle diffère radicalement
C-5	Injection de l'ego-motion IMU dans le suivi	Les traqueurs embarqués recensés n'exploitent pas l'odométrie déjà disponible

8. L'entonnoir de décision

Voici le raisonnement complet qui mène de l'état des lieux aux technologies retenues. Chaque étape élimine des options pour une raison explicite et opposable.

ÉTAT DES LIEUX : ce qui existe

Production : LiDAR + ultrasons + vision d'appoint, sécurité certifiée, ascenseur par réseau
Recherche : YOLOv7 + SAM2 + Grounding DINO sur GPU
Bas coût : YOLO seul sur Raspberry Pi, 6-8 FPS, sans structuration ni confiance



FILTRE 1 – Le matériel est imposé : Raspberry Pi 5, CPU nu

x Éliminé : tout ce qui suppose un GPU → SAM 2, Grounding DINO, cuRobo, DETR lourds
x Éliminé : les caméras de profondeur haut de gamme (hors budget, et aveugles au verre)
✓ Retenu : une classe de modèles « nano », < 5 M paramètres



FILTRE 2 – E2 : la sécurité relève du matériel certifié, pas de l'IA

→ L'IA n'a pas à être parfaite : elle doit être RÉGULIÈRE et HONNÊTE sur sa confiance
✓ Conséquence : le déterminisme de la latence (p95) devient un critère de premier rang
✓ Conséquence : la calibration de la confiance devient un livrable, pas un ornement



FILTRE 3 – E1 : la géométrie d'abord

x Éliminé : la profondeur monoculaire apprise (MiDaS, Depth Anything)
→ coût d'un second réseau, sortie non métrique, alors que le LiDAR donne la distance en métrique pour un coût CPU nul
x Éliminé : le segmenteur d'espace libre dédié (PP-LiteSeg, SegFormer-B0)
→ +30 à 80 ms et une campagne d'annotation pixel, alors que l'occupation LiDAR couvre l'essentiel du besoin
✓ Retenu : LiDAR 2D pour la distance et l'espace libre ; sonar pour le verre



FILTRE 4 – E3 : ne pas deviner ce qui est connu

x Éliminé : la classification visuelle de l'état de l'ascenseur comme source primaire
✓ Retenu : interface réseau ascenseur (à arbitrer avec l'encadrant), la vision en secours et en vérification de cohérence



FILTRE 5 – E4 : une seule dorsale dans la boucle

x Éliminé : tout empilement détection + segmentation + profondeur
✓ Retenu : un détecteur unique ; les autres fonctions en géométrie/statistique



FILTRE 6 – E5 : 6-8 FPS est le régime naturel, il faut le dépasser

→ L'effort porte sur la CHAÎNE D'EXÉCUTION, pas sur l'architecture
✓ Retenu : NCNN (×3 à ×5 sur ARM) + INT8 (×2,1 grâce au dot-product du Cortex-A76)
+ résolution réduite (×2,4) → objectif ≈ 25-30 ms



FILTRE 7 – Départage final entre modèles nano (grille §9 de l'état de l'art)

✓ YOLO26n (4,10/5) : sans NMS → latence déterministe, exigée par le filtre 2
↳ Repli 1 : YOLO11n (3,95/5), export NCNN plus éprouvé
↳ Repli 2 : NanoDet-Plus (3,00/5), frugalité extrême si le budget échoue



FILTRE 8 – E6 : ne pas réinventer le format de sortie

✓ Retenu : sortie structurée alignée sur le modèle en couches de Nav2 / ROS 2

9. Traçabilité des choix — de l'observation à la décision

Tableau de traçabilité complet : chaque technologie retenue renvoie à l'observation de l'existant qui la justifie. C'est la pièce à présenter en soutenance.

Choix technologique	Observation de l'existant	Enseignement	Alternative écartée	Motif de l'élimination
YOLO26n comme détecteur unique	Les prototypes RPi utilisent tous un YOLO nano ; les modèles lourds exigent un GPU	E4, E5	RF-DETR, D-FINE, SAM 2	Pas de chemin d'export ARM éprouvé ; conçus pour GPU
Sans NMS (déterminisme)	ISO 3691-4 impose un comportement prévisible pour les fonctions liées aux personnes	E2	YOLOv8n / YOLO11n (avec NMS)	Latence variable selon l'encombrement de la scène — au pire moment
NCNN + INT8 + 416 px	Le régime RPi observé (6-8 FPS) est sous la cible ; les revues embarquées convergent sur quantification + architectures légères	E5	PyTorch, ONNX FP32	Facteur ~5 laissé sur la table
LiDAR 2D pour la distance	TUG, HOSPI, Relay : tous fondés sur la télémétrie	E1	Profondeur monoculaire apprise	Doublerait le budget d'inférence pour une sortie non métrique
Sonar conservé	Les ultrasons survivent dans tous les systèmes commerciaux	E1	Suppression du sonar	Le verre est invisible au LiDAR et à la caméra — très présent en milieu hospitalier
Pas de segmenteur dédié	Aucun système en production n'embarque de segmentation sémantique dense pour l'espace libre	E1, E4	PP-LiteSeg, SegFormer-B0	+30-80 ms et annotation pixel par pixel, pour un besoin déjà couvert par le LiDAR
ByteTrack (géométrie)	Les prototypes RPi s'arrêtent à la détection, sans persistance temporelle	E4, E5	DeepSORT, BoT-SORT+ReID	Un CNN par piste : rédhibitoire sur CPU
Ego-motion par IMU	Les systèmes en production disposent tous d'odométrie ; BoT-SORT l'estime par vision faute de mieux	E1	Compensation visuelle du mouvement (CMC)	Estimer par vision ce qu'un capteur donne gratuitement
Confiance calibrée (temperature scaling)	Aucun prototype embarqué recensé ne quantifie son incertitude — c'est un vide	E2	Scores bruts ; MC-Dropout ; ensembles	Scores bruts : non calibrés, donc trompeurs. MC-Dropout/ensembles : coût x N passes
Ascenseur par réseau	HOSPI et Relay le font ainsi, tous les deux	E3	Classification visuelle de l'état	Information exacte disponible sans incertitude — la deviner serait une régression
Sortie alignée Nav2	La costmap en couches est le standard de facto du domaine	E6	Format propriétaire	Non réutilisable, non interopérable
CPU nu (Hailo-8L en repli documenté)	Les produits mettent le matériel nécessaire ; la question du sujet est justement la frugalité	—	AI Kit dès le départ	Répondrait à côté de la problématique posée (« frugale »)

10. Écarts assumés avec l'existant et risques associés

Tout écart avec une pratique établie doit être justifié et surveillé. Voici les nôtres.

Écart	Pratique du domaine	Notre choix	Justification	Risque	Parade
Pas de SLAM	Tous les systèmes en production font du SLAM	Perception locale, sans carte globale	Le sujet porte sur la perception, pas sur la localisation	La sortie « couloir dégagé » manque de contexte global	Interface Nav2 : la carte est fournie par l'aval
Calcul très en dessous	Production : x86 embarqué / GPU	Raspberry Pi 5, CPU nu	C'est la problématique même du sujet	Budget de latence non tenu	Repli documenté : Hailo-8L, résolution réduite
Pas de scanner de sécurité certifié	ISO 3691-4 : PLd obligatoire	Prototype de recherche, non certifiable	Hors périmètre d'un PRI	Sur-interprétation du niveau de sécurité atteint	À écrire explicitement dans le rapport final : le système est un démonstrateur de perception, pas un organe de sécurité
LiDAR 2D et non 3D	Le haut de gamme passe au 3D	2D	Coût, budget CPU, disponibilité	Aveugle hors du plan de balayage (obstacles en hauteur, surplombs)	Caméra + sonar couvrent partiellement ; à documenter comme limite connue
Jeu de données propre	Les industriels disposent d'années de données terrain	Collecte limitée	Contrainte de projet	Faible généralisation à d'autres sites	Pré-annotation par modèle enseignant ; augmentation de données agressive ; limite annoncée

11. Sources

Systèmes en production

- **Aethon — Intel RealSense dans les robots de livraison autonomes**
<https://www.intelrealsense.com/autonomous-mobile-robotics/>
- **How TUG Robots Are Revolutionizing Healthcare Logistics — AZoRobotics**
<https://www.azorobotics.com/Article.aspx?ArticleID=725>
- **Aethon TUG — présentation technique**
<https://www.roboticgizmos.com/aethon-tug-healthcare-robot/>
- **Panasonic — Autonomous Mobility Robot (technologies clés)**
<https://news.panasonic.com/global/stories/2019/69861.html>
- **Panasonic HOSPI — certification ISO 13482 et JIS**
<https://news.panasonic.com/global/topics/5001>
- **Panasonic HOSPI à Changi General Hospital — étude de cas**
<https://ap.connect.panasonic.com/th/en/case-studies/panasonic-autonomous-delivery-robots-hospi-aid-hospital-operations-changi-general>
- **Hospi — fiche Wikipedia**
<https://en.wikipedia.org/wiki/Hospi>
- **Relay Robotics — nouvelle gamme pour hôtels et hôpitaux**
<https://relayrobotics.com/relay-robotics-announces-new-line-of-delivery-robots-for-hotels-and-hospitals/>
- **Relay2 — intégration ascenseur (The Robot Report)**
<https://www.therobotreport.com/relay2-delivery-robot-offers-2x-payload-new-elevator-integration/>

- **Relay+ — génération suivante (The Robot Report)**
<https://www.therobotreport.com/relay-is-saviokes-new-generation-of-service-robot/>
- **Diligent Robotics — Moxi**
<https://www.diligentrobots.com/moxi>

Recherche académique

- **ORB: Operating Room Bot — Automating Operating Room Logistics through Mobile Manipulation (arXiv 2509.15600, IEEE CASE 2025)**
<https://arxiv.org/html/2509.15600>
- **Artificial Intelligence for Autonomous Mobile Robots in IR4.0-IR6.0: A Unified Review**
<https://doi.org/10.3390/machines14080950>
- **Edge AI in Practice: A Survey and Deployment Framework for Neural Networks on Embedded Systems (MDPI Electronics)**
<https://www.mdpi.com/2079-9292/14/24/4877>
- **ResNet-18 based multi-task visual inference and adaptive control for an edge-deployed autonomous robot (Frontiers in Robotics and AI, 2025)**
<https://www.frontiersin.org/journals/robotics-and-ai/articles/10.3389/frobt.2025.1680285/full>
- **AI-based approaches for improving autonomous mobile robot localization in indoor environments: a comprehensive review**
<https://www.sciencedirect.com/science/article/pii/S2215098625000321>
- **Deep Learning-Based Multi-Modal Fusion for Robust Robot Perception and Navigation (arXiv 2504.19002)**
<https://arxiv.org/pdf/2504.19002>
- **Deep Learning Perspective of Scene Understanding in Autonomous Robots (arXiv 2512.14020)**
<https://arxiv.org/pdf/2512.14020>

Projets embarqués bas coût

- **Development of an Autonomous Surveillance and Object Detection Robot using Raspberry Pi 5 and YOLOv8**
https://www.academia.edu/144419870/Project_Report_Development_of_of_an_Autonomous_Surveillance_and_Object_Detection_Robot_using_Raspberry_Pi_5_and_YOLOv8
- **Enabling Autonomous Navigation in Dynamic Environments: Affordance-Based Mobility (Springer)**
https://link.springer.com/chapter/10.1007/978-3-032-10667-4_7
- **Embedded Intelligent ADAS Car Prototype Using Raspberry Pi and YOLOv12n (Springer)**
https://link.springer.com/chapter/10.1007/978-3-032-18144-2_10
- **Design of a Raspberry Pi 4B-Based Guide Robot with YOLO-v10 Integration**
https://www.researchgate.net/publication/393581912_Design_of_a_Raspberry_Pi_4B-Based_Guide_Robot_with_YOLO-v10_Integration
- **Optimizing object detection for autonomous robots: a comparative analysis of YOLO models (Measurement)**
<https://www.sciencedirect.com/science/article/abs/pii/S0263224125020354>
- **An Embedded Computer Vision Approach to Environment Modeling and Local Path Planning in AMRs**
<https://www.sciencedirect.com/org/science/article/pii/S1526149225004722>

Briques logicielles et normes

- **Nav2 — Costmap 2D, configuration des couches**
<https://navigation.ros.org/configuration/packages/configuring-costmaps.html>
- **nav2_costmap_2d — vue d'ensemble du paquet ROS**
https://index.ros.org/p/nav2_costmap_2d/
- **Nav2 — filtrage des obstacles induits par le bruit**
https://docs.nav2.org/tutorials/docs/filtering_of_noise-induced_obstacles.html
- **ISO 3691-4:2020 — Chariots sans conducteur et leurs systèmes**
<https://www.iso.org/standard/70660.html>
- **Mobile Robot Safety Standards: ISO 3691-4 et ANSI/RIA R15.08**
[https://blog.saphira.ai/mobile-robot-safety-standards-understanding-iso-3691-4-\(driverless-industrial-trucks\)-and-r15-08-\(industrial-mobile-robots\)-implementation](https://blog.saphira.ai/mobile-robot-safety-standards-understanding-iso-3691-4-(driverless-industrial-trucks)-and-r15-08-(industrial-mobile-robots)-implementation)

- **ISO 3691-4 : le cadre de responsabilité pour les AGV**
<https://www.agvnetwork.com/automated-guided-vehicles-technology/standard-3691-4>
 - **The Latest in Autonomous Mobile Robots: New Safety Standards (A3 / Automate)**
<https://www.automate.org/robotics/industry-insights/autonomous-mobile-robot-safety-updates-new-features>
-

État de l'art et comparaison des modèles

PRI 2026-2027 — Projet 2 : IA embarquée pour la perception des véhicules autonomes en logistique hospitalière
 Livrable : « État de l'art et comparaison des modèles » — version 1, septembre 2026

1. Méthodologie de la comparaison

1.1 Principe

Comparer « des modèles d'IA » dans l'absolu n'a pas de sens ici. La comparaison est menée **brique fonctionnelle par brique fonctionnelle** (F1 à F5, cf. cadrage §3 (partie I)), selon la grille imposée par le cahier des charges :

Famille (cahier des charges)	Métriques opérationnelles retenues
Performance — précision, robustesse, complexité	mAP@50-95, mAP@50, rappel classe « personne » , dégradation en basse lumière, nombre de paramètres / GFLOPs
Frugalité — taille, RAM/CPU, énergie	Taille du fichier déployé (Mo), RSS du processus (Mo), puissance moyenne (W)
Embarqué — temps d'inférence, CPU, temps réel	Latence médiane et p95 sur RPi 5 (ms), % CPU, cadence soutenue (Hz), déterminisme

Trois critères supplémentaires, non listés explicitement mais **décisifs pour ce projet**, sont ajoutés et justifiés :

- **Transférabilité** — capacité du modèle à être ré-entraîné sur un petit jeu de données hospitalier. Les classes utiles (brancard, chariot de soins, pied à perfusion, porte d'ascenseur) sont absentes de COCO : un modèle performant sur COCO mais avide de données en ré-entraînement est inutilisable ici.
- **Calibration de la confiance** — le cahier des charges exige explicitement la sortie « niveau de confiance / incertitude » (sortie *f*).
- **Maturité de l'outillage embarqué** — existence d'un export ARM CPU testé (NCNN, TFLite, ONNX Runtime). Une architecture sans chemin d'export propre coûte des semaines de projet.

1.2 Convention de lecture des chiffres

⚠ Avertissement méthodologique. Les latences publiées dans la littérature et sur les blogs techniques sont **très dispersées** pour un même modèle : selon la résolution d'entrée (320 / 416 / 640), le nombre de threads, le format d'export, la version de l'OS (32 vs 64 bits) et surtout le refroidissement, un même YOLO nano sur Raspberry Pi 5 est rapporté entre ~80 ms et ~260 ms. Les tableaux ci-dessous servent donc à **classer les candidats et à éliminer**, jamais à dimensionner. Les valeurs définitives sont produites par le protocole du §11, sur le matériel du projet.

Chaque valeur porte un indicateur de confiance :

Marque	Signification
	Publiée dans l'article de référence du modèle (benchmark COCO officiel)
	Rapportée par une source secondaire (documentation d'éditeur, benchmark communautaire) — ordre de grandeur
	Estimée par extrapolation dans ce document — à confirmer par mesure
	Non documentée pour cette configuration — à mesurer

2. F1 — Détection d'objets 2D

C'est la brique centrale : elle produit les sorties *a* (obstacle / position / distance), *b* (personne présente) et contribue à *d* (ascenseur disponible).

2.1 Les quatre familles architecturales en présence

a) Détecteurs mono-étage à ancrs, dorsale mobile (2017-2020)

SSD-MobileNetV2 / SSDLite, EfficientDet-Lite0/1/2. Historiquement la référence de l'embarqué. Principe : une dorsale à convolutions séparables en profondeur (*depthwise separable*) + des têtes de détection sur plusieurs échelles.

- **Ce qui a vieilli** : la précision. EfficientDet-Lite0/1/2 plafonne autour de **23 / 28 / 32 mAP**  sur COCO — c'est-à-dire *en dessous* d'un YOLO nano moderne qui atteint ~40 mAP pour une empreinte comparable.
- **Ce qui reste** : un outillage TFLite INT8 exceptionnellement mature et une compatibilité totale avec les accélérateurs (Coral, NPU de smartphone).
- **Verdict pour ce projet : écartés comme candidat principal.** Un écart de 8 à 17 points de mAP se paie directement en faux négatifs sur des personnes, ce qui est le risque à ne pas prendre. Conservés comme référence basse (*baseline*) dans le protocole expérimental.

b) Détecteurs mono-étage sans ancrs, ultra-légers

NanoDet / NanoDet-Plus. Conçu dès l'origine pour le mobile : tête FCOS sans ancrs, module d'assignation guidée (AGM), assignateur à étiquettes douces dynamiques (DSLAs) et pyramide légère Ghost-PAN. NanoDet-Plus revendique **+7 mAP** sur COCO par rapport à NanoDet, pour **980 Ko en INT8 / 1,8 Mo en FP16** et ~97 FPS sur smartphone .

- **Ce qui séduit** : c'est, et de loin, le modèle le plus frugal du tableau — un ordre de grandeur sous les YOLO nano en taille de fichier.
- **Ce qui inquiète** : projet communautaire, **peu maintenu depuis 2022**, écosystème d'entraînement et d'export moins robuste, très peu de retours de déploiement récents.
- **Verdict : candidat de repli sérieux** si et seulement si le budget de latence s'avère intenable pour les YOLO nano sur CPU nu. Le risque projet (outillage) est réel mais la frugalité est inégalée.

c) Détecteurs YOLO modernes (v8 → v11 → 26)

C'est la famille dominante de l'embarqué en 2026. Points clés de l'évolution :

Version	Année	Apport structurant pour l'embarqué
YOLOv5n / YOLOv8n	2020 / 2023	Sans ancres (v8), export NCNN/TFLite mature, énorme base de retours d'expérience
YOLOv10n	2024	Introduction de l'entraînement sans NMS (double assignation)
YOLO11n	2024	Meilleur compromis précision/paramètres à empreinte égale ; YOLO11x atteint 54,7 mAP avec 56,9 M paramètres , contre 54,3 mAP et 76 M paramètres pour RT-DETRv2-x 📄
YOLO26n	janvier 2026	Conçu explicitement pour l'edge : bout-en-bout sans NMS , suppression de la <i>Distribution Focal Loss</i> , ProgLoss + STAL (petits objets). 2,4 M paramètres, 40,9 mAP COCO 🌐 ; l'éditeur annonce jusqu'à +43 % de vitesse d'inférence CPU 🌐

L'apport le plus important pour ce projet n'est pas le gain de mAP, c'est la suppression du NMS. Le *Non-Maximum Suppression* est un post-traitement dont le coût dépend du **nombre de détections** dans l'image : il est rapide dans un couloir vide, lent dans un hall bondé — exactement le moment où la perception doit être la plus réactive. Un modèle sans NMS a une **latence déterministe**, ce qui satisfait directement le critère C4 (latence au 95^e centile) du cadrage. Pour un système soumis à une exigence de temps réel en présence de personnes, c'est un argument de sécurité, pas de confort.

d) Transformeurs de détection temps réel

RT-DETR / RT-DETRv2 / RT-DETRv4, D-FINE, RF-DETR. Ces architectures ont supprimé le principal défaut des DETR (convergence lente) et dominant aujourd'hui le haut du classement COCO :

- D-FINE-M : **52,3 mAP** contre 51,5 mAP pour YOLO11-M [📄](#)
- RF-DETR (dorsale DINOv2, Apache 2.0, ICLR 2026) : première famille temps réel à dépasser **60 AP sur COCO** [📄](#) ; la déclinaison **RF-DETR-Nano** est annoncée à **2,3 ms / 67,6 AP50** [📄](#) — mais **sur GPU T4**, pas sur CPU ARM.
- **Ce qui séduit** : la meilleure précision absolue, et surtout une **excellente transférabilité** (RF-DETR est explicitement conçu pour le *fine-tuning* sur petits jeux de données de domaine, et domine le classement RF100-VL d'adaptabilité — précisément notre cas d'usage hospitalier).
- **Ce qui bloque** : l'attention globale est **mal servie par un CPU ARM**. Les gains mesurés sur GPU (parallélisme massif, TensorRT) ne se transposent pas ; les opérateurs d'attention sont mal optimisés dans NCNN/TFLite, et la quantification INT8 des transformeurs reste délicate. Aucun benchmark RPi 5 fiable n'est disponible [?](#).
- **Verdict** : **écartés pour l'inférence embarquée sur CPU nu**, mais **retenus comme modèle enseignant** pour une distillation de connaissances vers le modèle embarqué, et comme **auto-annotateur** du jeu de données hospitalier (§11.2). C'est la bonne façon d'exploiter leur précision sans en payer le coût à l'exécution.

2.2 Tableau comparatif — référence COCO

Base commune d'évaluation (COCO val2017, 640×640 sauf mention). Ces chiffres **ne dimensionnent pas** le système ; ils classent les candidats.

Modèle	Type	Params (M)	mAP@50-95	Sans NMS	Licence	Confiance
SSD-MobileNetV2	Ancres, CNN	~4,3	~22	non	Apache 2.0	🌐
EfficientDet-Lite0	Ancres, CNN	~3,2	~23	non	Apache 2.0	🌐
EfficientDet-Lite2	Ancres, CNN	~5,2	~32	non	Apache 2.0	🌐
NanoDet-Plus-m	Sans ancres, CNN	~1,2	~27	non	Apache 2.0	🌐
YOLOv8n	Sans ancres, CNN	3,2	37,3	non	AGPL-3.0	📄
YOLO11n	Sans ancres, CNN	2,6	39,5	non	AGPL-3.0	📄
YOLO26n	Sans ancres, CNN	2,4	40,9	oui	AGPL-3.0	🌐
RF-DETR-Nano	Transformeur	~-	67,6 @50	oui	Apache 2.0	📄
D-FINE-M	Transformeur	~19	52,3	oui	Apache 2.0	📄
RT-DETRv2-x	Transformeur	76	54,3	oui	Apache 2.0	📄

🚩 **Point de vigilance licence.** Les modèles Ultralytics (YOLOv8 / YOLO11 / YOLO26) sont sous **AGPL-3.0**. Pour un PRI académique publié en dépôt ouvert, c'est compatible. Si le projet devait déboucher sur une valorisation industrielle fermée, une licence commerciale serait nécessaire — et **RF-DETR / D-FINE / NanoDet (Apache 2.0) deviendraient structurellement préférables**. À signaler à l'encadrant : ce n'est pas un détail, c'est un critère de choix à part entière.

2.3 Tableau comparatif — comportement rapporté sur Raspberry Pi 5

Modèle	Format	Résolution	Latence rapportée	Cadence	Verdict vs budget 50 ms	Confiance
YOLOv8n	ONNX, multi-thread	640	~300 ms	~3,4 FPS	🔴	🌐
YOLOv8n	NCNN , multi-thread	640	~258 ms	~9,0 FPS*	🔴	🌐
YOLO11n	PyTorch	640	~400 ms	~2,5 FPS	🔴	🌐
YOLO11n	NCNN	640	~80 ms	~12 FPS	🟡	🌐
YOLO11n	ONNX	640	~147 ms	6,79 FPS	🟡	🌐
YOLO26n	ONNX	640	~128 ms	7,79 FPS	🟡	🌐
YOLO26n	NCNN	640	~67 ms	~15 FPS	🟡	🌐
YOLO26n	NCNN INT8	416	~25-35 ms	~30-40 FPS	🟢	📊
NanoDet-Plus-m	NCNN INT8	416	~20-30 ms	~35-50 FPS	🟢	📊
RF-DETR-Nano	ONNX	640	non documenté	—	—	?
YOLOv8s	Hailo-8L (AI Kit)	640	~12-40 ms	25-80 FPS	🟢	🌐

* Les sources 🌐 sur YOLOv8n NCNN sont contradictoires (9,0 FPS annoncés pour 258 ms de latence moyenne — incohérence probablement due à un pipeline en parallèle ou à des conditions de mesure différentes). C'est exactement pourquoi le §11 impose une re-mesure interne.

2.4 Lecture du tableau — les trois enseignements

1. **Le format d'export pèse plus lourd que le choix du modèle.** Passer de PyTorch à NCNN sur YOLO11n fait passer la latence de ~400 ms à ~80 ms, soit **un facteur 5**. Aucun changement d'architecture dans ce tableau n'offre un tel gain. **Conclusion : l'effort d'optimisation (§7) est prioritaire sur l'effort de sélection d'architecture.**
2. **Aucun modèle ne tient le budget de 50 ms en 640×640 sur CPU nu.** Le budget est atteignable uniquement en combinant **NCNN + INT8 + réduction de résolution à 416 ou 320**. C'est la décision d'ingénierie centrale du projet, et elle doit être validée par la mesure, pas par la littérature.
3. **L'écart YOLO26n / YOLO11n (~15 % de vitesse, +1,4 mAP) est réel mais secondaire** face au facteur 5 du point 1. YOLO26n est néanmoins préféré pour une raison qualitative : le **déterminisme** apporté par l'absence de NMS.

2.5 Résolution d'entrée : le levier le plus rentable

Le coût de calcul d'un CNN varie approximativement avec le **carré** de la résolution :

Résolution	Coût relatif	Latence estimée (base YOLO26n NCNN @640 ≈ 67 ms)	Effet attendu sur la détection
640×640	1,00	~67 ms 🌐	Référence
512×512	0,64	~43 ms 📊	Perte faible sur objets moyens/proches
416×416	0,42	~28 ms 📊	Perte notable sur petits objets lointains
320×320	0,25	~17 ms 📊	Perte forte ; personnes lointaines manquées

Argument spécifique au cas d'usage : dans un couloir hospitalier, les obstacles pertinents (personne, chariot, brancard) sont **grands dans l'image** parce que proches — c'est la géométrie d'un couloir de 2 à 3 m de large. La pénalité de la réduction de résolution y est donc bien plus faible que sur COCO, dont la métrique est dominée par les petits objets. **Ce point doit être vérifié expérimentalement sur le jeu de données du projet** — c'est l'expérience la plus rentable à mener en premier (§11.3).

3. F2 — Segmentation de l'espace libre

Cette brique produit les sorties *c* (couloir dégagé ou obstrué) et *e* (trajectoire libre ou bloquée).

3.1 Pourquoi la détection par boîtes ne suffit pas

Un détecteur répond à « qu'y a-t-il et où ? ». Il ne répond pas à « **puis-je passer ?** ». Trois cas où F1 échoue seul, tous fréquents en milieu hospitalier :

- une **flaque** ou un sol mouillé signalé — pas un objet, mais une zone non navigable ;
- une **marche ou un dénivelé** (seuil de porte, quai de livraison) — invisible en boîte 2D ;
- un obstacle d'une **classe non apprise** — le détecteur ne produit aucune boîte, et le couloir est déclaré libre à tort. **C'est le mode de défaillance le plus dangereux du système.**

La segmentation de l'espace libre répond à la question inverse et complémentaire : elle identifie **le sol navigable** plutôt que les obstacles. Un obstacle inconnu devient alors simplement « une zone qui n'est

pas du sol ». C'est un filet de sécurité par construction.

3.2 Candidats

Modèle	Type	Argument principal	Limite pour RPi 5
BiSeNetV2	CNN deux branches	Architecture conçue pour le temps réel ; ~82,3 % mIoU sur jeux de données de référence 🌐	Deux branches = deux passes ; coûteux sur CPU
PP-LiteSeg	CNN encodeur-décodeur léger	Jusqu'à 77,5 % mIoU sur Cityscapes avec un très bon compromis vitesse 🌐 ; outillage PaddlePaddle mature	Écosystème PaddlePaddle, export ARM moins courant que NCNN
SegFormer-B0	Transformeur + décodeur MLP	~83,0 % mIoU, comparable à BiSeNetV2 🌐 ; très bonne robustesse	Même problème d'attention sur CPU ARM que les DETR
YOLO*n-seg (segmentation d'instances)	Mono-étage	Mutualise la dorsale avec F1 — une seule inférence pour détection + masques	mAP de masque plus faible qu'un segmenteur dédié
Fusion géométrique LiDAR 2D	Non appris	Coût CPU quasi nul , robuste, déterministe, ne nécessite aucune annotation	2D seulement : aveugle aux obstacles hors du plan de balayage, et au verre

3.3 Recommandation pour F2

Ne pas ajouter un second réseau de neurones. Deux options sont défendables, la seconde est retenue :

- ❌ **Segmenteur dédié (PP-LiteSeg / SegFormer-B0)** : ajoute 30 à 80 ms de latence 📊 sur un budget déjà tendu, et exige une campagne d'annotation pixel par pixel — la plus coûteuse qui soit.
- ✅ **Variante -seg du détecteur retenu, activée uniquement si le budget de latence le permet après optimisation, complétée par une occupation géométrique issue du LiDAR 2D.** Le LiDAR fournit gratuitement une carte d'occupation locale fiable ; le réseau n'a alors à traiter que ce que le LiDAR ne voit pas (verre, obstacles bas hors plan, sols mouillés).

C'est le meilleur rapport valeur/coût : le filet de sécurité « obstacle non classifié » est assuré par un capteur géométrique déterministe, pas par un réseau supplémentaire.

4. F3 — Estimation de la distance

Le cahier des charges exige « obstacle détecté / **position / distance** » : il faut passer du pixel au mètre.

4.1 Comparaison des trois voies

Approche	Précision métrique	Coût CPU	Robustesse hospitalière	Annotation requise
Profondeur monoculaire apprise (MiDaS-Small, Depth Anything V2-Small)	Relative, non métrique sans calibration	Élevé — un second réseau complet	Sensible aux sols réfléchissants	Aucune (pré-entraîné)
Caméra RGB-D / stéréo	Métrique, ~cm	Faible (calcul déporté dans le capteur)	Échoue sur le verre et les surfaces brillantes	Aucune
LiDAR 2D + projection de la boîte	Métrique, ~6 cm d'erreur moyenne rapportés 🌐	Quasi nul	Excellente, sauf verre	Aucune

4.2 Ce que dit la littérature sur la profondeur monoculaire embarquée

Les modèles de profondeur monoculaire ont fait des progrès spectaculaires en qualité, mais restent **hors budget sur CPU ARM** :

- Depth Anything V2-Small : **20,99 ms sur GPU T4** 📄 — soit, par extrapolation des ratios observés sur les détecteurs, plusieurs centaines de millisecondes sur RPi 5 🇬🇧.
- MiDaS v2.1-Small a été déployé sur Raspberry Pi 4 (TFLite + OpenCV, C++) 🌐, et PyD-Net atteint ~2 Hz sur Raspberry Pi 3 🌐. Les ordres de grandeur sont clairs : **c'est une charge comparable à celle du détecteur lui-même**, pour une sortie *relative* qu'il faut ensuite calibrer.

Conclusion : la profondeur monoculaire apprise est écartée. Elle doublerait le budget d'inférence pour fournir une information que le LiDAR donne mieux, en métrique, et gratuitement.

4.3 Recommandation pour F3 — fusion géométrique caméra / LiDAR 2D

L'approche retenue est la **fusion par secteur angulaire** (*Angular Sector Fusion*), documentée dans la littérature récente sur les robots d'intérieur : pour chaque boîte produite par le détecteur, on projette son secteur angulaire sur le balayage LiDAR et on prend la distance par percentile bas des retours du secteur.

- Méthode **déterministe, non apprise, à coût CPU négligeable** — elle n'entame pas le budget.
- La littérature rapporte, avec un filtrage de Kalman, une **erreur moyenne de 0,06 m**, avec un taux de succès de **95 % sur obstacles statiques** mais seulement **60 % sur obstacles dynamiques** 🌐. Ce dernier chiffre est un **avertissement direct pour notre cas d'usage** : les personnes en mouvement sont précisément des obstacles dynamiques. D'où l'importance de la brique F4 (suivi), qui apporte la cohérence temporelle manquante.
- Contrainte d'installation à retenir : le LiDAR doit être monté **bas** (~16 cm du sol dans les travaux cités) pour intercepter les objets usuels d'intérieur (personnes, chaises, plantes, pieds de chariot).

Le sonar conserve un rôle propre et non substituable : les portes vitrées et les sas, très présents en milieu hospitalier, sont invisibles à la fois au LiDAR (le faisceau traverse) et à la caméra (transparence). C'est la justification technique de sa présence dans le parc de capteurs.

5. F4 — Suivi multi-objets

Le suivi transforme des détections image par image en **objets persistants dotés d'une identité et d'une vitesse**. Il conditionne les sorties e (trajectoire libre ou bloquée — impossible sans vitesse estimée) et f (niveau de confiance — une détection confirmée sur N images est bien plus fiable qu'une détection isolée).

5.1 Comparaison

Traqueur	Principe	MOTA (MOT17)	IDF1 (MOT17)	Coût CPU	Réidentification apprise
SORT	Kalman + IoU	~74	~62	Négligeable	non
ByteTrack	Association en deux passes, y compris détections à faible score	80,3 	77,3 	Négligeable	non
OC-SORT	SORT à mise à jour centrée observation	~78	~77	Négligeable	non
BoT-SORT	ByteTrack + compensation du mouvement caméra (CMC) + apparence	80,5 	80,2 	Modéré à élevé (CMC + extracteur ReID)	oui (variante)
DeepSORT	Kalman + descripteur d'apparence appris	~61	~62	Élevé (un CNN par piste)	oui

5.2 Analyse et recommandation

ByteTrack est retenu. Justification :

1. **Coût CPU négligeable** : purement géométrique (Kalman + IoU), aucun réseau supplémentaire. La littérature rapporte un ensemble YOLOv8n + ByteTrack à **plus de 108 FPS**  — le traqueur ne consomme pratiquement rien à côté du détecteur.
2. **Son principe fondateur sert directement notre contrainte de sécurité.** ByteTrack associe *aussi* les détections à faible score de confiance, au lieu de les jeter. Une personne partiellement occultée par un brancard produit typiquement une détection à faible score : ByteTrack la conserve, un traqueur classique la perd. **C'est un gain direct sur le rappel « personne »**, notre critère C1.
3. **BoT-SORT est meilleur (+2,9 IDF1) mais pour de mauvaises raisons ici** : son gain vient surtout de la compensation du mouvement caméra, conçue pour des séquences filmées à la main ou en panoramique. Sur un VA, **l'IMU et l'odométrie fournissent l'ego-motion directement et gratuitement** — il est inutile de l'estimer par vision. La variante ReID coûte un CNN par piste : rédhibitoire.

 **Contribution d'ingénierie proposée** : injecter l'ego-motion IMU/odométrie dans la prédiction du filtre de Kalman de ByteTrack. On obtient l'essentiel du bénéfice de BoT-SORT **à coût nul**, en exploitant des capteurs déjà présents au cahier des charges. C'est un point différenciant à mettre en avant dans le rapport final.

6. F5 — Fusion, état sémantique et niveau de confiance

6.1 Sorties d'état sémantique (c, d, e)

Ces sorties ne sont **pas** produites par un réseau de neurones, mais par une **logique de décision alimentée par F1-F4**. C'est un choix d'architecture délibéré : une règle explicite est auditable, déterministe et ne demande aucune annotation, là où un classifieur d'état exigerait un jeu de données dédié pour un gain nul.

Sortie	Logique proposée
c — couloir dégagé / obstrué	Aucune piste confirmée dans le corridor de navigation ET occupation LiDAR libre sur $\geq X$ m
d — ascenseur disponible	Détection « porte d'ascenseur » + état (ouverte/fermée) confirmé sur N images + absence de personne dans l'ouverture
e — trajectoire libre / bloquée	Projection des pistes F4 avec leur vitesse sur l'horizon de temps d'arrêt ; test d'intersection avec le corridor

La sortie *d* mérite une remarque : « ascenseur disponible » n'est **pas** une classe d'objet, c'est un **état composite**. Le détecteur fournit « porte d'ascenseur » et éventuellement « indicateur lumineux » ; l'état se déduit ensuite. Confondre les deux conduirait à créer des classes artificielles (« ascenseur-disponible », « ascenseur-occupé ») très coûteuses à annoter et fragiles.

6.2 Le niveau de confiance (sortie f) — la vraie difficulté

Le score brut d'un détecteur **n'est pas une probabilité**. Les réseaux modernes sont notoirement **sur-confiants** : un lot de détections à 0,9 est empiriquement juste à ~70 %. Livrer ce score tel quel à l'IA de décision serait une faute : la décision ferait confiance à une mesure non calibrée.

Chaîne de production du niveau de confiance proposée :

```
score brut du détecteur
  ↳ 1. Calibration (Temperature Scaling sur un jeu de validation dédié)
    ↳ 2. Agrégation temporelle (âge de la piste, régularité des associations, ByteTrack)
      ↳ 3. Accord inter-capteurs (la boîte est-elle confirmée par le LiDAR ? le sonar)
        ↳ 4. Indicateur de conditions (luminosité, flou de mouvement, T° CPU)
          ↳ confiance finale  $\in [0,1]$ , calibrée ( $ECE \leq 0,05$ , critère C9)
```

- **Temperature scaling** : un seul paramètre scalaire appris *après* l'entraînement, coût d'inférence strictement nul. C'est le meilleur rapport efficacité/coût de la littérature sur la calibration.
- **Vérification par diagramme de fiabilité** et *Expected Calibration Error* — à intégrer au protocole expérimental. C'est un livrable à part entière, explicitement demandé par le sujet.
- Les méthodes plus lourdes (MC-Dropout, ensembles profonds) sont **écartées** : elles multiplient le coût d'inférence par le nombre de passes, ce qui est incompatible avec le budget.

7. Chaîne d'exécution : formats, runtimes et optimisation

Le §2.4 l'a établi : **c'est ici que se gagne la faisabilité du projet**, plus que dans le choix d'architecture.

7.1 Comparaison des runtimes d'inférence sur ARM

Runtime	Points forts	Points faibles	Adapté au projet
PyTorch	Référence pour l'entraînement	Non destiné à l'inférence embarquée ; ~400 ms sur YOLO11n @640 🌐	Entraînement seulement
ONNX Runtime	Universel, très bon support des opérateurs, INT8 dynamique et statique	Moins optimisé que NCNN sur ARM pour les CNN	Oui — référence de portabilité
NCNN	Le plus rapide sur CPU ARM (Tencent, conçu pour le mobile) ; facteur ~5 mesuré vs PyTorch 🌐	Couverture d'opérateurs plus étroite (problématique pour les transformeurs)	Oui — cible de déploiement
TFLite	Noyaux INT8 ARM optimisés à la main ; écosystème mature ; indispensable pour Coral	Conversion depuis PyTorch indirecte et parfois fragile	Oui — alternative INT8
OpenVINO	Excellent sur x86 (≈ ×3,3 en INT8 🌐)	Cible Intel — sans objet sur RPi 5	Non

7.2 Quantification INT8 : pourquoi elle fonctionne précisément sur le Pi 5

C'est un point technique qui mérite d'être explicité dans le rapport, car il est **spécifique au matériel imposé** :

Le gain de l'INT8 dépend entièrement de la présence d'une instruction de **produit scalaire** dans le jeu d'instructions (ARM *dotprod* / x86 *VNNI*). La littérature de mesure est sans ambiguïté : sur **Cortex-A76 (Raspberry Pi 5), qui possède les instructions dot-product, l'INT8 apporte environ ×2,1** 🌐. Sur un **Cortex-A53 (Raspberry Pi 3), qui ne les possède pas, l'INT8 est ~1,76× plus lent** que le FP32 🌐.

Conséquence directe : le choix du Raspberry Pi 5 dans le cahier des charges n'est pas neutre — c'est la génération à partir de laquelle la stratégie INT8 devient payante. D'anciens travaux mesurent jusqu'à **×4 en TFLite INT8 sur Raspberry Pi** par rapport au FP32 🌐. La perte de précision reste faible **à condition que la calibration soit soignée** (jeu de calibration représentatif : couloirs de jour *et* de nuit, plusieurs sites).

7.3 Techniques d'optimisation, classées par rapport gain/effort

Technique	Gain attendu	Effort	Risque	Priorité
Export NCNN	×3 à ×5 🌐	Faible	Faible	1
Réduction de résolution (640 → 416)	×2,4 🏠	Très faible	Perte sur petits objets — à mesurer	2
Quantification INT8	×2,1 🌐	Moyen (jeu de calibration)	Perte de mAP si calibration bâclée	3
Élagage structuré (<i>pruning</i>)	×1,2 à ×1,5	Élevé (ré-entraînement)	Perte de précision	4
Distillation (RF-DETR → YOLO26n)	+1 à +3 mAP à coût d'inférence nul	Élevé	Aucun à l'exécution	5 (si le temps le permet)
Réduction du nombre de classes	×1,05 à ×1,1	Nul (vient du jeu de données)	Aucun	Systématique

Gain cumulé estimé : $\times 3$ (NCNN) $\times 2,4$ (416) $\times 2,1$ (INT8) $\approx \times 15$ par rapport à la référence PyTorch @640 🇸🇮. Partant de ~ 400 ms, cela place la cible autour de **25-30 ms**, c'est-à-dire **dans le budget de 50 ms**, avec de la marge pour F2-F5.

 Les gains ne se multiplient jamais exactement (saturation mémoire, surcoûts de quantification/déquantification). L'estimation $\times 15$ est un **majorant optimiste** ; un facteur $\times 8$ à $\times 10$ serait déjà un succès. C'est l'objet de la mesure §11.

8. Option matérielle : accélérateurs pour Raspberry Pi 5

À traiter comme **plan de repli**, pas comme solution de premier rang — le sujet demande de démontrer la **frugalité**, et une solution qui n'existe que grâce à un accélérateur à 70 € répond moins bien à la problématique posée.

Solution	Puissance de calcul	Performance rapportée	Consommation	Remarque
CPU seul (4× A76)	—	cf. §2.3	7-12 W système	Cible principale : c'est la vraie démonstration de frugalité
AI Kit / AI HAT+ (Hailo-8L)	13 TOPS	YOLOv8s @640 : 25-35 FPS , jusqu'à 80 FPS en PCIe Gen 3 🌐	3-4 TOPS/W ; système < 4 W en 320×320 @ 5 FPS 🌐	Occupe le port PCIe ; nécessite le SDK Hailo et une compilation HEF du modèle
Google Coral USB	4 TOPS	Bon en TFLite INT8	~ 2 W	Contraintes d'opérateurs fortes ; écosystème en perte de vitesse

Position recommandée : mener le projet sur **CPU nu**, et documenter le Hailo-8L comme **extension mesurée** dans le protocole expérimental. Cela donne deux points de comparaison dans le rapport final (frugalité maximale vs performance maximale) et transforme un risque en résultat.

9. Grille de décision multicritère

Pondérations dérivées des critères d'évaluation du cahier des charges (les trois familles y pèsent à parts comparables), avec une pondération renforcée sur le déterminisme et la transférabilité, justifiée aux §2.1c et §1.1.

Critère	Poids	SSD-MobileNetV2	NanoDet-Plus	YOLOv8n	YOLO11n	YOLO26n	RF-DETR-N
Précision (mAP, rappel personne)	20 %	1	2	3	4	4	5
Latence sur RPi 5 CPU	20 %	4	5	3	4	4	1
Frugalité (taille, RAM)	15 %	4	5	3	4	4	2
Déterminisme (sans NMS)	10 %	1	1	1	1	5	5
Maturité de l'export ARM	15 %	5	3	5	5	4	1
Transférabilité (petit jeu de données)	10 %	2	2	4	4	4	5
Écosystème & maintenance	10 %	3	1	5	5	4	4
Score pondéré / 5		2,95	3,00	3,40	3,95	4,10	3,05

Échelle : 1 = inadapté, 5 = excellent. Le détail du calcul est reproductible à partir du tableau.

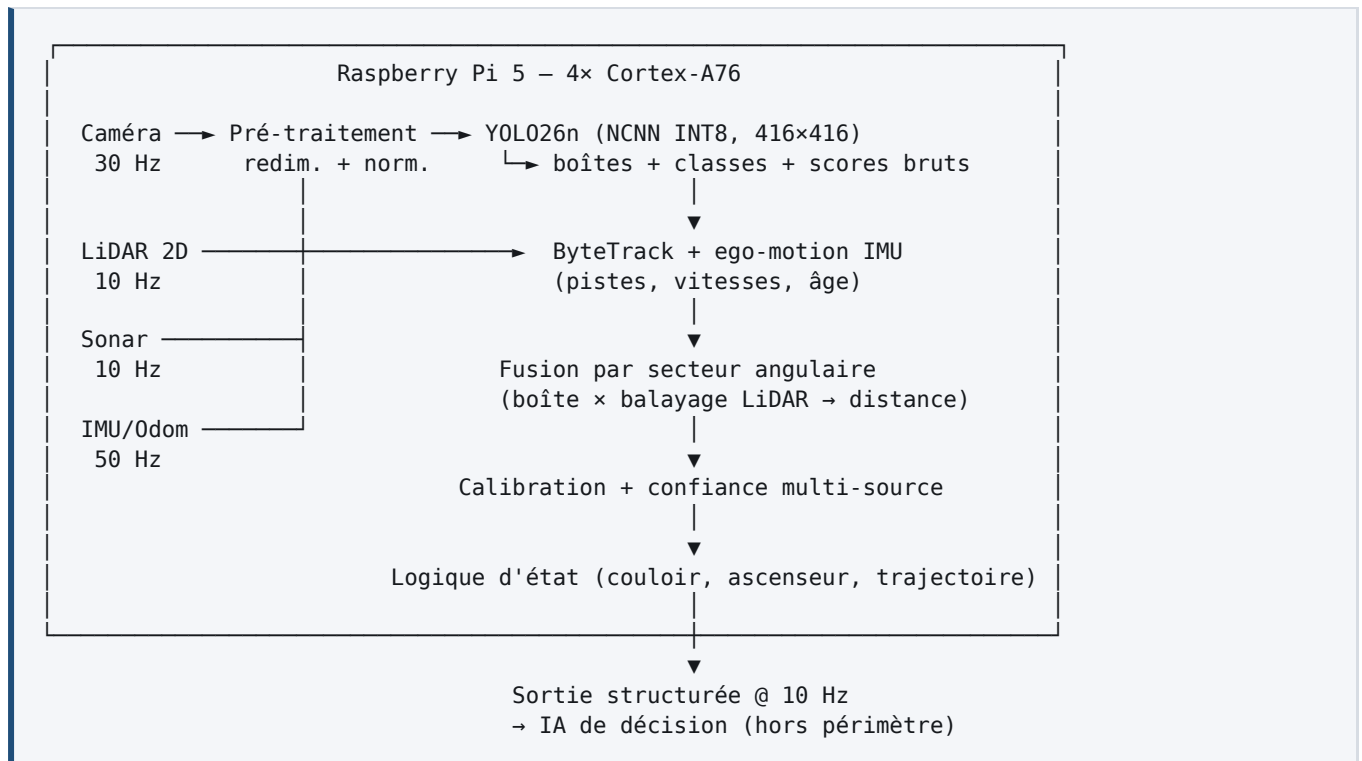
Lecture des résultats :

- **YOLO26n arrive en tête (4,10)** grâce au cumul d'un bon niveau sur tous les critères et d'un avantage exclusif sur le déterminisme.
- **YOLO11n le suit de près (3,95)** avec un écosystème d'export plus éprouvé — c'est le **repli immédiat** si YOLO26 pose des difficultés d'export NCNN, son principal risque (modèle récent, janvier 2026).
- **NanoDet-Plus (3,00)** domine sur la frugalité pure ; il reste le repli en cas d'échec du budget de latence.
- **RF-DETR-Nano (3,05)** obtient les meilleures notes de précision et de transférabilité mais est pénalisé par l'absence de chemin d'export ARM éprouvé. **Son score bas ne reflète pas sa qualité, mais son inadéquation à cette cible matérielle précise** — d'où son rôle d'enseignant/annotateur.

⚠ **Ce classement repose partiellement sur des estimations** 🌐/🇫🇷. Il oriente le choix ; il ne le clôt pas. Les trois premiers candidats doivent être re-mesurés selon le §11 avant que le livrable « choix argumenté du modèle » ne soit figé.

10. Recommandation argumentée

10.1 Architecture de perception proposée



10.2 Synthèse des choix

Brique	Choix retenu	Repli	Argument décisif
F1 Détection	YOLO26n , NCNN INT8, 416×416	YOLO11n (export plus éprouvé) → NanoDet-Plus (frugalité)	Meilleur score multicritère ; latence déterministe grâce à l'absence de NMS
F2 Espace libre	Occupation LiDAR 2D + variante -seg si le budget le permet	Segmenteur dédié PP-LiteSeg	Évite un second réseau et une campagne d'annotation pixel
F3 Distance	Fusion par secteur angulaire caméra/LiDAR 2D	Caméra RGB-D	Métrique, ~6 cm d'erreur, coût CPU nul ; la profondeur monoculaire doublerait le budget
F3' Verre / sas	Sonar	—	Seul capteur voyant les surfaces vitrées
F4 Suivi	ByteTrack + ego-motion IMU	OC-SORT	Coût nul ; conserve les détections à faible score → gain de rappel sur « personne »
F5 Confiance	Temperature scaling + agrégation temporelle + accord inter-capteurs	—	Coût d'inférence nul ; répond à la sortie <i>f</i> du sujet
Runtime	NCNN (référence ONNX Runtime conservée)	TFLite INT8	Facteur ~5 mesuré sur ARM
Matériel	CPU nu , Hailo-8L documenté en extension	AI Kit si le budget de latence échoue	La frugalité est un critère d'évaluation du sujet

10.3 Ce qui rend cette architecture cohérente avec la problématique

Le sujet demande une IA « **performante, frugale et apte au déploiement embarqué** ». La logique suivie tient en une phrase : **un seul réseau de neurones dans la boucle**.

Toutes les autres fonctions (distance, suivi, état, confiance) sont assurées par des méthodes géométriques ou statistiques à coût quasi nul, qui exploitent des capteurs déjà présents au cahier des charges. C'est ce qui permet de tenir simultanément les trois familles de critères d'évaluation — là où un empilement de réseaux spécialisés (détection + segmentation + profondeur) saturerait le CPU dès la première démonstration.

11. Protocole de validation expérimentale

Ce qui précède est un état de l'art : il **oriente**, il ne prouve pas. Voici les mesures à produire pour transformer cette recommandation en « choix argumenté » défendable.

11.1 Banc de mesure sur Raspberry Pi 5

Conditions imposées (sans quoi la mesure n'est pas reproductible) :

- Raspberry Pi OS 64 bits, dissipateur **actif**, alimentation officielle 27 W
- **10 minutes de chauffe** avant toute mesure ; relevé de `vcgencmd measure_temp` et `vcgencmd get_throttled` en continu
- 1 000 images, les 50 premières écartées (préchauffage des caches)
- Report de la **médiane**, du **p95** et du **maximum** — pas seulement de la moyenne
- Wattmètre USB-C en ligne pour la puissance système
- `psutil` pour le RSS et le pourcentage CPU par cœur

Matrice d'essais (3 modèles × 3 formats × 3 résolutions = 27 configurations) :

Axe	Valeurs
Modèle	YOLO26n · YOLO11n · NanoDet-Plus-m
Format	PyTorch (référence) · ONNX FP32 · NCNN FP16 · NCNN INT8
Résolution	640 · 416 · 320

11.2 Jeu de données

- **Collecte terrain** : couloirs, halls d'ascenseur, sas, de jour et de nuit, avec et sans circulation. Anonymisation par floutage des visages **à la source** (RGPD, secret médical).
- **Classes proposées** : `personne`, `personne_au_sol`, `chariot`, `brancard`, `pied_a_perfusion`, `porte_ascenseur`, `porte`, `obstacle_statique`.
- **Pré-annotation par RF-DETR ou D-FINE** (exécuté hors ligne sur machine de bureau, pas sur le Pi), puis correction manuelle. C'est ainsi que la précision des transformeurs est exploitée sans en payer le coût embarqué.
- **Jeu de test dédié basse lumière**, séparé, pour mesurer le critère de robustesse.

11.3 Expérience prioritaire — la courbe résolution / rappel

C'est l'expérience à mener en premier, avant tout entraînement long. Elle conditionne toute l'architecture.

Tracer, sur le jeu de test hospitalier : **rappel sur la classe « personne » en fonction de la résolution d'entrée (640 / 512 / 416 / 320)**, croisé avec la distance de l'objet.

- Si le rappel se maintient à 416 (hypothèse du §2.5 : les obstacles sont proches donc grands dans l'image en couloir), **la faisabilité du projet sur CPU nu est acquise**.
- S'il s'effondre, il faut arbitrer immédiatement entre : réduire la vitesse du VA, accepter 5-7 Hz, ou basculer sur l'accélérateur Hailo-8L. **Le savoir en semaine 4 plutôt qu'en semaine 20 est la meilleure réduction de risque du projet.**

11.4 Mesures de calibration

Diagramme de fiabilité et *Expected Calibration Error* avant / après temperature scaling, sur le jeu de validation. Cible : **ECE $\leq$ 0,05** (critère C9 du cadrage).

12. Sources

Détection d'objets — architectures et benchmarks

- Ultralytics YOLO26 — modèle edge-first, NMS-free
<https://www.ultralytics.com/blog/ultralytics-yolo26-the-new-standard-for-edge-first-vision-ai>
- Ultralytics YOLO26 — documentation du modèle
<https://docs.ultralytics.com/models/yolo26>
- YOLO26: An Analysis of NMS-Free End to End Framework for Real-Time Object Detection (arXiv 2601.12882)
<https://arxiv.org/pdf/2601.12882>
- Ultralytics YOLO Evolution: YOLO26, YOLO11, YOLOv8, YOLOv5 (arXiv 2510.09653)
<https://arxiv.org/pdf/2510.09653>
- YOLO11 vs RT-DETRv2 — comparaison
<https://docs.ultralytics.com/compare/yolo11-vs-rtdetr>
- RT-DETR — DETRs Beat YOLOs on Real-time Object Detection (arXiv 2304.08069)
<https://arxiv.org/pdf/2304.08069>
- RT-DETRv4 (arXiv 2510.25257)
<https://arxiv.org/pdf/2510.25257>
- RF-DETR: Neural Architecture Search for Real-Time Detection Transformers (arXiv 2511.09554)
<https://arxiv.org/pdf/2511.09554>
- RF-DETR Nano / Small / Medium — annonce Roboflow
<https://blog.roboflow.com/rf-detr-nano-small-medium/>
- Best Object Detection Models 2026 — RF-DETR, YOLOv12 & Beyond
<https://blog.roboflow.com/best-object-detection-models/>
- NanoDet-Plus — dépôt de référence
<https://github.com/Awaker1/NanoDet-Plus>
- A Comprehensive Evaluation of Deep Learning Object Detection Models on Heterogeneous Edge Devices (arXiv 2409.16808)
<https://arxiv.org/html/2409.16808>
- Computer Vision Model Leaderboard — Roboflow
<https://leaderboard.roboflow.com/>

Raspberry Pi 5 — mesures et accélération

- **YOLO on Raspberry Pi — setup & benchmarks (Ultralytics)**
<https://docs.ultralytics.com/guides/raspberry-pi>
- **RPi YOLO Benchmark — YOLOv8n (NCNN vs ONNX, multi-thread)**
<https://kadirertabatay.github.io/rpi-yolo-benchmark/benchmarks/yolov8/yolov8n/>
- **YOLO26 on a Raspberry Pi 5 with the AI Kit — NCNN export & frame-rate benchmarks**
<https://aegisai.in/yolo26-raspberry-pi-5-ai-kit-hailo-ncnn-benchmarks/>
- **Benchmark on RPi5 and CM4 running YOLOv8s with the RPi AI Kit — Seeed Studio**
https://wiki.seeedstudio.com/benchmark_on_rpi5_and_cm4_running_yolov8s_with_rpi_ai_kit/
- **Testing Raspberry Pi's AI Kit — 13 TOPS (Jeff Geerling)**
<https://www.jeffgeerling.com/blog/2024/testing-raspberry-pis-ai-kit-13-tops-70/>
- **Computer Vision Applications with YOLO — Edge AI Engineering**
https://mjrovai.github.io/EdgeML_Made_Ease_ebook/raspi/object_detection/cv_yolo.html

Quantification et runtimes

- **Is INT8 Portable? A Cross-Platform Measurement Study of Quantized Inference (arXiv 2609.16085)**
<https://arxiv.org/html/2609.16085>
- **Performance Characterization of using Quantization for DNN Inference (arXiv 2303.05016)**
<https://arxiv.org/pdf/2303.05016>
- **Accelerating Deep Learning Model Inference on Arm CPUs with Ultra-Low Bit Quantization (arXiv 2207.08820)**
<https://arxiv.org/pdf/2207.08820>
- **Does Quantization Improve Inference Speed? It Depends**
<https://par.nsf.gov/servlets/purl/10636811>

Segmentation et profondeur

- **PP-LiteSeg: A Superior Real-Time Semantic Segmentation Model (arXiv 2204.02681)**
<https://arxiv.org/abs/2204.02681>
- **Real-Time Freespace Segmentation on Autonomous Robots (arXiv 1902.00842)**
<https://arxiv.org/pdf/1902.00842>
- **Depth-guided Free-space Segmentation for a Mobile Robot (arXiv 2311.01966)**
<https://arxiv.org/html/2311.01966>
- **Enhancing Robustness of Indoor Robotic Navigation with Free-Space Segmentation Models (arXiv 2402.08763)**
<https://arxiv.org/pdf/2402.08763>
- **MiDaS v2.1-small sur Raspberry Pi (TFLite + OpenCV, C++)**
<https://github.com/KozhaAkhmet/MiDaS-v2.1-small-cpp>
- **Monocular Depth Estimation — Ultralytics**
<https://docs.ultralytics.com/tasks/depth>

Suivi multi-objets

- **SORT vs ByteTrack vs OC-SORT vs BoT-SORT vs C-BIoU — MOT Benchmark Comparison**
<https://trackers.roboflow.com/latest/trackers/comparison/>
- **BoT-SORT: Robust Associations Multi-Pedestrian Tracking (arXiv 2206.14651)**
<https://arxiv.org/pdf/2206.14651>
- **Observation-Centric SORT (arXiv 2203.14360)**
<https://arxiv.org/pdf/2203.14360>
- **Multi-Object Tracking — DeepSORT, ByteTrack, OC-SORT in production 2026**
<https://www.forasoft.com/learn/ai-for-video-engineering/articles-ai/multi-object-tracking-deepsort-bytetrack-ocsort>
- **IndoorCrowd: A Multi-Scene Dataset for Human Detection, Segmentation, and Tracking (arXiv 2604.02032)**
<https://arxiv.org/pdf/2604.02032>

Fusion capteurs et robotique d'intérieur

- **Lightweight Semantic-Aware Route Planning on Edge Hardware: Monocular Camera-2D LiDAR Fusion (PMC13075264)**
<https://pmc.ncbi.nlm.nih.gov/articles/PMC13075264/>
 - **Fusion of 2D LiDAR and Vision-Based Detection for Collision-Aware Indoor Navigation (Springer)**
https://link.springer.com/chapter/10.1007/978-3-032-18474-0_26
 - **A Review of Sensing Technologies for Indoor Autonomous Mobile Robots (PMC10893033)**
<https://pmc.ncbi.nlm.nih.gov/articles/PMC10893033/>
 - **Sensor Data Fusion for a Mobile Robot Using Neural Networks (PMC8749872)**
<https://www.ncbi.nlm.nih.gov/pmc/articles/PMC8749872/>
 - **LiDAR vs RGB-D Cameras for AMRs — Orbbec**
<https://www.orbbec.com/blog/how-lidar-and-rgbd-cameras-compare-and-work-together/>
-

Document de travail — version 1. Les valeurs marquées 🌐 et 🇪🇺 doivent être confirmées par les mesures du §11 avant la remise du livrable « choix argumenté du modèle ».